TD2 Conteneurs, Dockerfile et Docker Compose

Module : Réseaux : cloud et Internet des objets — Master 2 RSA

Savoir-faire

- Instancier un conteneur, le manipuler et rendre ses données persistantes avec un volume.
- Publier un service avec un port et monter un dossier de l'hôte.
- Écrire un Dockerfile et construire une image.
- Décrire une application multi-services dans un fichier Compose.
- Relier des services par un réseau et se préparer au déploiement de la plateforme du projet.

Rappel :

Notion	À retenir
Image	Modèle en lecture seule, composé de couches
Conteneur	Instance en cours d'exécution d'une image, avec une couche en lecture/écriture éphémère
Volume	Stockage persistant, indépendant du cycle de vie du conteneur
Réseau	Par défaut le bridge `docker0` ; sur un réseau défini par l'utilisateur, les conteneurs se joignent par leur nom
Compose	Décrit plusieurs services, réseaux et volumes dans un fichier YAML

Obtenir de l'aide : `docker --help`, `docker run --help`, `docker compose --help`.

Environnement de travail

- Linux (machine virtuelle ou WSL2) avec Docker installé : `sudo apt install -y docker.io docker-compose-v2`, ou l'installation par le dépôt officiel Docker. Vérifiez : `docker --version` et `docker compose version`.
- Dans ce TP, la commande Compose est `docker compose` (avec un espace). L'ancienne commande `docker-compose` (avec un tiret) n'est plus maintenue.
- Pour éviter de taper `sudo` devant chaque commande : `sudo usermod -aG docker $USER`, puis rouvrez votre session.

Compte rendu. Conservez vos commandes et les captures des résultats demandés

Exercice 1 : Cycle de vie d'un conteneur et volumes

Objectif : comprendre qu'un conteneur est éphémère et à quoi servent les volumes.

1. Lancez un conteneur Alpine interactif :

```
docker run -ti --name c1 alpine
```

1. Dans le conteneur, créez un dossier et des fichiers :

```
mkdir R508 && cd R508 && touch f1 f2 f3 && ls
exit
```

2. Dans le terminal de l'hôte, listez les conteneurs actifs puis tous les conteneurs :

```
docker ps
docker ps -a
```

3. Redémarrez le conteneur et rattachez-vous à lui :

```
docker start -ai c1
ls R508
exit
```

4. Supprimez le conteneur, recréez-le avec le même nom et regardez :

```
docker rm c1
docker run -ti --name c1 alpine ls /R508
```

5. Créez un volume et utilisez-le dans deux conteneurs successifs :

```
docker volume create mydata
docker run --rm -v mydata:/data alpine sh -c "echo bonjour > /data/message.txt"
docker run --rm -v mydata:/data alpine cat /data/message.txt
docker volume inspect mydata
```

Questions

1. Après le point 3, les fichiers existent-ils encore ? Pourquoi ?
2. Après le point 4, que se passe-t-il ? Quelle est la différence avec le point 3 ?
3. Que fait l'option `--rm` ?
4. Dans le point 5, le second conteneur voit-il le fichier ? Où le volume est-il stocké sur l'hôte (champ `Mountpoint`) ?

Ce qu'il faut comprendre : un conteneur est éphémère : sa couche en écriture disparaît avec lui. Ce qui doit survivre (base de données, configuration, mesures) se met dans un volume.

Exercice 2 : Un serveur web Nginx avec port et volume

Objectif : publier un service et faire évoluer son contenu sans reconstruire quoi que ce soit.

1. Créez le dossier du site :

```
mkdir -p ~/www
```

2. Lancez Nginx en publiant le port 80 du conteneur sur le port 8080 de l'hôte, avec le dossier monté :

```
docker run -d --name monsitedocker -p 8080:80 -v ~/www:/usr/share/nginx/html nginx
```

3. Ouvrez `http://localhost:8080` dans un navigateur, ou avec `curl -i http://localhost:8080`.
4. Créez une page d'accueil et actualisez :

```
echo "<h1>Bienvenue sur mon site hébergé par Nginx dans Docker</h1>" > ~/www/index.html
```

5. Affichez les journaux et les propriétés du conteneur :

```
docker logs monsitedocker
docker inspect monsitedocker
```

5. Relevez l'adresse IP du conteneur dans la section `Networks`.
6. Observez le réseau : `ip a` (interface `docker0`), `docker network inspect bridge`.

Questions

1. Que constatez-vous au point 3 tant que le dossier `~/www` est vide ?
2. Que constatez-vous après le point 4, sans redémarrer le conteneur ? En quoi est-ce utile en développement ?
3. Que montrent les journaux après vos accès ? Que représente chaque ligne ?
4. Quelle est l'adresse IP du conteneur ? Dans quel sous-réseau se trouve-t-elle ? Faites le lien avec `docker0` du TP précédent.
5. Pouvez-vous accéder au site avec l'adresse IP de l'hôte et le port 8080 depuis la machine d'un camarade ? De quoi cela dépend-il ?

Ce qu'il faut comprendre : `-p hôte:conteneur` ouvre un chemin entre l'extérieur et le conteneur, `-v hôte:conteneur` partage un dossier. Le conteneur n'est pas modifié : le site vit dans le dossier de l'hôte.

Exercice 3 : Construire sa propre image (Dockerfile)

Objectif : empaqueter une petite API Flask dans une image.

1. Créez un dossier de travail `vols/` et placez-y les deux fichiers suivants.

1. `app.py` :

```
from flask import Flask, jsonify

app = Flask(__name__)

vols = [
    {"id": 1, "destination": "Paris", "prix": 150},
    {"id": 2, "destination": "New York", "prix": 400},
    {"id": 3, "destination": "Tokyo", "prix": 600},
]


@app.route("/vols")
def liste_vols():
    return jsonify(vols)


if __name__ == "__main__":
    app.run(host="0.0.0.0", port=5000)
```

1. `requirements.txt` :

```
flask
```

2. Complétez le fichier `Dockerfile` :

```
# Image de base officielle, légère
FROM python:3.12-slim
# Dossier de travail dans le conteneur
WORKDIR ..........
# Copier d'abord les dépendances, pour profiter du cache de construction
COPY ..........
# Installer les dépendances
RUN ..........
# Copier ensuite le code de l'application
COPY ..........
# Port sur lequel l'application écoute
EXPOSE ..........
# Commande de démarrage
CMD ["python", "app.py"]
```

3. Construisez l'image, lancez-la et testez :

```
docker build -t api-vols .
docker run -d --name api -p 5000:5000 api-vols
curl http://localhost:5000/vols
```

4. Affichez l'historique des couches : `docker image history api-vols`.

5. Modifiez `app.py` (ajoutez un vol), reconstruisez l'image et observez le temps et les messages de construction.

Questions

1. À quoi sert le point (`.`) à la fin de `docker build` ?
2. Pourquoi copie-t-on `requirements.txt` avant le reste du code ?
3. Quelle est la différence entre `EXPOSE` et l'option `-p` de `docker run` ?
4. Combien de couches l'image contient-elle ? Quelle est sa taille (`docker images`) ?
5. Pourquoi ne faut-il jamais écrire un mot de passe dans un Dockerfile ?

Ce qu'il faut comprendre : un Dockerfile décrit comment fabriquer une image de façon reproductible ; l'ordre des instructions détermine l'efficacité du cache.

Exercice 4 : Compose, une application Flask et PostgreSQL

Objectif : décrire et lancer une application à deux services avec un fichier Compose.

1. Créez un dossier `appdb/` avec la structure suivante :

```
appdb/
├── app.py
├── requirements.txt
├── Dockerfile
├── compose.yaml
├── .env
└── .gitignore
```

2. `requirements.txt` :

```
flask
psycopg2-binary
```

3. `app.py` : cette application compte les visites dans une base PostgreSQL.

```
import os
import time

import psycopg2
from flask import Flask, jsonify

app = Flask(__name__)


def connexion():
    return psycopg2.connect(
        host=os.environ.get("DB_HOST", "db"),
        dbname=os.environ["POSTGRES_DB"],
        user=os.environ["POSTGRES_USER"],
        password=os.environ["POSTGRES_PASSWORD"],
    )


def executer(requete):
    conn = connexion()
    try:
        with conn, conn.cursor() as cur:
            cur.execute(requete)
            return cur.fetchone() if cur.description else None
    finally:
        conn.close()


def initialiser_base():
    for _ in range(10):
        try:
            executer("CREATE TABLE IF NOT EXISTS visites (id SERIAL PRIMARY KEY, date TIMESTAMP
DEFAULT now())")
            return
        except psycopg2.OperationalError:
            time.sleep(2)
    raise RuntimeError("Base de données indisponible")


@app.route("/")
def accueil():
    executer("INSERT INTO visites DEFAULT VALUES")
    total = executer("SELECT count(*) FROM visites")[0]
    return jsonify(message="Bonjour depuis un conteneur", visites=total)


@app.route("/sante")
def sante():
    return jsonify(statut="ok")


if __name__ == "__main__":
    initialiser_base()
    app.run(host="0.0.0.0", port=5000)
```

4. `Dockerfile` : reprenez celui de l'exercice 3 (le port est `5000`, la commande `python app.py`).

5. `.env` (variables lues par Compose) et `.gitignore` :

```
POSTGRES_DB=mydatabase
POSTGRES_USER=postgres
POSTGRES_PASSWORD=choisissez-un-mot-de-passe
```

.env

5. Choisissez votre propre mot de passe. Le fichier `.env` ne doit **jamais** être publié dans un dépôt Git.

6. `compose.yaml` : complétez les parties marquées `À COMPLÉTER`.

```
services:
  app:
    build: .                        # construire l'image depuis le Dockerfile local
    ports:
      - "À COMPLÉTER"               # publier le port 5000 sur le port 5000 de l'hôte
    environment:
      DB_HOST: db
      POSTGRES_DB: ${POSTGRES_DB}
      POSTGRES_USER: ${POSTGRES_USER}
      POSTGRES_PASSWORD: ${POSTGRES_PASSWORD}
    depends_on:
      db:
        condition: service_healthy # attendre que la base réponde
    networks:
      - À COMPLÉTER                 # réseau app_network

  db:
    image: postgres:16
    environment:
      POSTGRES_DB: ${POSTGRES_DB}
      POSTGRES_USER: ${POSTGRES_USER}
      POSTGRES_PASSWORD: ${POSTGRES_PASSWORD}
    volumes:
      - À COMPLÉTER:/var/lib/postgresql/data   # volume nommé data_db
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U $${POSTGRES_USER} -d $${POSTGRES_DB}"]
      interval: 5s
      timeout: 3s
      retries: 10
    restart: always
    networks:
      - À COMPLÉTER                 # réseau app_network

volumes:
  À COMPLÉTER:                      # déclarer le volume data_db

networks:
  À COMPLÉTER:                      # déclarer le réseau app_network
```

7. Lancez et testez :

```
docker compose up --build -d
docker compose ps
curl http://localhost:5000/
curl http://localhost:5000/
docker compose logs app
```

8. Testez la persistance :

```
docker compose down
docker compose up -d
curl http://localhost:5000/
```

8. Puis supprimez aussi le volume : `docker compose down -v`.

Questions

1. Combien de services contient le fichier ? Sur quel port l'application est-elle accessible ?
2. Comment l'application trouve-t-elle la base de données ?
3. Le compteur de visites continue-t-il après `docker compose down` puis `up` ? Et après `down -v` ? Pourquoi ?

4. Pourquoi les variables sont-elles dans `.env` et non dans le fichier Compose ? Que se passe-t-il si vous publiez `.env` dans Git ?

6. Le service `db` publie-t-il un port sur l'hôte ? Pourquoi est-ce une bonne pratique ?

Ce qu'il faut comprendre : Compose crée un réseau par projet, où chaque service est joignable par son nom. Les volumes conservent les données, les variables se séparent du code, et la base n'a pas besoin d'être exposée à l'extérieur.

Exercice 5 : Un premier échange MQTT, aperçu de votre projet

Objectif : déployer en Compose un broker MQTT, un abonné et un « capteur » simulé. C'est une version réduite de la plateforme de votre projet.

1. Créez un dossier `mqtt/` avec deux fichiers.

1. `mosquitto.conf` :

```
listener 1883
allow_anonymous true
persistence true
persistence_location /mosquitto/data/
log_dest stdout
```

1. `compose.yaml` :

```
services:
  broker:
    image: eclipse-mosquitto:2
    volumes:
      - ./mosquitto.conf:/mosquitto/config/mosquitto.conf:ro
      - mosquitto_data:/mosquitto/data
    ports:
      - "1883:1883"
    networks:
      - iot

  abonne:
    image: eclipse-mosquitto:2
    command: mosquitto_sub -h broker -t "campus/#" -v
    depends_on:
      - broker
    networks:
      - iot

  capteur:
    image: eclipse-mosquitto:2
    entrypoint: ["sh", "-c", "i=0; while true; do i=$$((i+1)); mosquitto_pub -h broker -t
campus/batiment1/energie -m $$i; sleep 2; done"]
    depends_on:
      - broker
    networks:
      - iot

volumes:
  mosquitto_data:

networks:
  iot:
```

2. Lancez la pile et observez les messages reçus par l'abonné :

```
docker compose up -d
docker compose logs -f abonne
```

3. Depuis l'hôte, ouvrez un second abonné dans le conteneur du broker :

```
docker compose exec broker mosquitto_sub -t "campus/#" -v
```

4. Publiez un message à la main :

```
docker compose exec broker mosquitto_pub -t campus/batiment1/alerte -m "seuil depasse"
```

5. Observez les réseaux et les conteneurs :

```
docker network ls
docker network inspect mqtt_iot
docker compose ps
```

5. Le nom du réseau est préfixé par celui du dossier : adaptez-le si votre dossier ne s'appelle pas `mqtt`.

Questions

1. Pourquoi `abonne` et `capteur` joignent-ils le broker avec le nom `broker` ?
2. Le port `1883:1883` du broker est-il nécessaire pour que `abonne` et `capteur` fonctionnent ? Pour quoi est-il utile ?
3. Quel est le rôle du volume `mosquitto_data` ?
4. Que signifie `allow_anonymous true` ? Quel risque cela représente-t-il pour votre plateforme ? *(vous le corrigerez en séance 5)*
5. Quels services de votre projet reconnaissez-vous dans cette pile, et lesquels manquent-ils (traitement, base de données, tableau de bord) ?

Ce qu'il faut comprendre : cette pile contient déjà les éléments du jalon J2 de votre projet : plusieurs services, un réseau commun, des volumes, des communications vérifiées. Il suffit d'y ajouter Node-RED, InfluxDB et Grafana.

Synthèse

Remplissez le tableau ci-dessous et conservez-le.

Action	Commande
Télécharger une image	
Lister les images	
Supprimer une image	
Créer et démarrer un conteneur	
Arrêter un conteneur	
Lister les conteneurs actifs	
Lister tous les conteneurs	
Afficher les journaux d'un conteneur	
Ouvrir un terminal dans un conteneur	
Publier un port	
Monter un dossier ou un volume	
Construire une image	
Démarrer un projet Compose en arrière-plan	
Voir l'état des services Compose	
Arrêter et supprimer les services, sans les volumes	
Arrêter et supprimer les services et les volumes	

Astuce : pour supprimer en masse les conteneurs dont le nom contient `c` : `docker rm -f $(docker container ls -q -f "name=c")`. Vérifiez la liste avant de supprimer.

Nettoyage

```
docker compose down -v          # dans chaque dossier de projet Compose
docker rm -f monsitedocker api c1
docker volume rm mydata
docker image rm api-vols
```

Les commandes signalent une erreur pour un élément qui n'existe pas : ce n'est pas grave.